

Benefícios e Limites da Programação Paralela Aplicada a um Simulador de Gerenciamento de Memória

Alunos: Fernando Infantini, Giovana Piazza, Ronaldy Gelos
Instituição: Universidade Federal do Pampa (Unipampa) - Campus Bagé

Julho de 2026

Resumo

Este artigo apresenta uma análise empírica de desempenho entre abordagens sequenciais e concorrentes para o gerenciamento dinâmico de memória (*heap*) em Java. O objetivo principal é avaliar o impacto da granularidade de tarefas e o custo de sincronização sob diferentes capacidades de hardware. Por meio de simulações com cargas massivas e um algoritmo *Best-Fit* otimizado, mapeou-se o comportamento do simulador sob estresse. Os experimentos revelaram que o ganho de eficiência inicia-se imediatamente a partir de 2 threads em todos os cenários. Contudo, a escalabilidade real mostrou-se estritamente dependente do tamanho da *heap*: enquanto estruturas pequenas (30 KB) sofrem com a saturação prematura devido ao baixo custo de processamento, cenários com alta carga de busca (500 KB) atingiram um *speedup* máximo de 2,57x com o platô ideal em 6 threads, convergindo exatamente com os limites físicos de núcleos do processador utilizado.

1 Introdução

O desenvolvimento de softwares para sistemas multiprocessados modernos exige a capacidade de utilizar eficientemente os múltiplos núcleos de processamento disponíveis no hardware. No contexto de Sistemas Operacionais, o gerenciamento dinâmico de memória (*heap*) é frequentemente um gargalo crítico de desempenho. Em aplicações baseadas em múltiplas threads (*multithreading*), a alocação concorrente introduz o desafio clássico da exclusão mútua: se várias threads modificarem os índices de um alocador simultaneamente, ocorrerão condições de corrida (*race conditions*), o que corrompe a integridade da estrutura de dados global.

Para mitigar esse problema, o uso tradicional de primitivas de sincronização, como monitores e exclusões mútuas (*mutexes*), é o padrão adotado pela maioria dos ambientes de execução. Contudo, travar regiões críticas inteiras cria um ponto de serialização rígido, limitando o paralelismo real da aplicação, de acordo com o que é previsto pela Lei de Amdahl. Quando o tempo gasto com a sincronização supera o tempo de processamento útil, o sistema apresenta um decréscimo severo de performance. Para contornar essa barreira, este estudo adota uma técnica de Leitura paralela, separando a busca linear pelo espaço livre de memória da gravação efetiva dos dados.

Este estudo utiliza um simulador de alocação customizado sob a política *Best-Fit*, munido de rotinas de expulsão aleatória e compactação contígua de dados. O principal objetivo é identificar de que forma a variação controlada no contingente de threads impacta o tempo total de execução e determinar onde residem as barreiras físicas (decorrentes da topologia do hardware) e lógicas (decorrentes do design de software) que impedem a escalabilidade contínua. O artigo está estruturado em: Introdução (Seção 1), Fundamentação Teórica (Seção 2), Trabalhos Relacionados (Seção 3), Arquitetura do Simulador (Seção 4), Avaliação Experimental (Seção 5) e Conclusões (Seção 6).

2 Fundamentação Teórica

2.1 Política de Alocação Best-Fit e Fragmentação

A alocação contígua exige que cada bloco de memória ocupe um espaço linear ininterrupto dentro do vetor representativo. O algoritmo *Best-Fit* inspeciona os blocos livres disponíveis e seleciona o menor bloco capaz de suprir a requisição em andamento, visando preservar fragmentos maiores para requisições futuras de alta magnitude [1]. A desvantagem dessa estratégia é o custo computacional da varredura, possuindo complexidade de tempo de pior caso $O(N)$ em arrays não ordenados.

Com a execução contínua de alocações e liberações, o alocador sofre de fragmentação externa: há memória total suficiente para novas alocações, porém dividida em pequenos subespaços pulverizados e não contíguos [2]. O sistema reage aplicando o conceito de compactação, deslocando todos os blocos ativos para o início da *heap*. Essa operação possui complexidade aritmética alta para a CPU, pois exige cópias sequenciais de vetores, invalidações de memória cache e a atualização em cadeia de mapas de ponteiros.

2.2 Sincronização de Threads e Leitura Paralela

Na linguagem Java, cada objeto possui um bloqueio intrínseco associado (*monitor lock*) [3]. Ao declarar uma região ou método como **synchronized**, apenas uma thread pode obter este monitor por vez, forçando todas as demais competidoras ao estado de bloqueio absoluto e induzindo o sistema operacional a realizar operações dispendiosas de troca de contexto (*context switching*).

Neste projeto, com o intuito de mitigar o gargalo de concorrência na busca, aplica-se o conceito de Controle de Concorrência Paralela (*Parallel Concurrency Control*). Sob este paradigma, as threads efetuam a leitura do estado geral do vetor de memória de forma livre de travas. Elas realizam a varredura linear de forma paralela e independente. Apenas no instante final de gravação do bloco é que a exclusão mútua é invocada, validando se o espaço previamente mapeado continua livre. Caso tenha sido alterado por outra thread, a operação é abortada e o ciclo de busca é reiniciado.

3 Trabalhos Relacionados

Vários estudos na literatura tentam resolver o problema de contenção na memória, propondo arquiteturas diferentes para evitar o uso exagerado de travas na programação concorrente.

Berger et al. [4] criaram o Hoard, um alocador muito conhecido que diminui esse gargalo dividindo a memória em várias partes (uma *heap* isolada para cada thread), usando a *heap* global só quando falta espaço. Comparando com os nossos resultados, o particionamento do Hoard escala muito melhor em cenários com dezenas de threads, já que ele quase elimina as colisões na hora de gravar. Porém, a nossa abordagem com leitura paralela em uma única *heap* foca em economizar espaço e evitar fragmentação. Nosso simulador obteve um ganho de velocidade (*speedup*) de 2,57x no cenário de 500 KB, sem desperdiçar tanta memória quanto as abordagens isoladas por thread.

Outra alternativa são os alocadores totalmente sem travas (*lock-free*), o trabalho de Michael [5], que usam instruções diretas do processador, como o *Compare-And-Swap* (CAS). Nos nossos testes com a *heap* de 100 KB, o tempo piora a partir de 8 threads, por causa de muitas colisões simultâneas. Os algoritmos *lock-free* não sofrem tanto com isso e se mantêm mais estáveis sob estresse. Mesmo assim, o nosso modelo (que faz a busca livre e a gravação com **synchronized**) se mostrou uma solução de ótimo custo-benefício para processadores comerciais de 6 núcleos, com uma implementação mais simples.

Mais recentemente, alocadores usados na indústria, como o *mimalloc* [6] desenvolvido pela Microsoft, melhoraram essa ideia de divisão quebrando as listas de memória livre em pedaços menores (*sharding*). Isso otimiza bastante o uso do cache nos processadores atuais. Embora o nosso simulador seja bem mais simples, a Leitura paralela que aplicamos segue o mesmo princípio central: tirar a sincronização pesada do processo de busca para deixar a CPU trabalhar de forma mais paralela e eficiente.

4 Arquitetura e Implementação do Simulador

4.1 Modelagem Estrutural da Heap

A memória simulada é construída de maneira independente das estruturas internas da Máquina Virtual Java (JVM), utilizando um vetor primitivo unidimensional `int[] heap`. Onde o valor zero denota ausência de dados (espaço livre) e valores inteiros positivos representam o identificador único (ID) da variável residente. Os metadados operacionais são gerenciados por um mapeamento indexado do tipo `HashMap`, responsável pelo rastreamento em tempo real do índice físico inicial e da extensão do bloco em bytes.

4.2 Mecanismo de Alocação Paralelo

A arquitetura do gerenciador concorrente remove o bloqueio global do método de busca, dividindo o processo em duas etapas bem definidas através de um laço iterativo de tentativa (*retry loop*):

1. **Busca Paralela:** A thread lê as referências de blocos livres no vetor *heap* sem reter travas lógicas, aproveitando de forma isolada a capacidade computacional de múltiplos núcleos do processador.
2. **Validação em Região Crítica:** Ao eleger um espaço candidato que atenda ao critério *Best-Fit*, a thread requisita a trava (`synchronized`). Uma validação pontual de consistência é feita. Se o espaço continuar livre, os dados são persistidos; caso contrário, a trava é liberada e um novo ciclo de busca paralela é disparado.

Essa abordagem impede que múltiplas threads executem varreduras sequenciais pesadas sob exclusão mútua. Quando o simulador detecta lotação absoluta da memória, a thread coordenadora entra em uma sub-região de contenção isolada, realizando a expulsão forçada de 30% das variáveis alocadas por meio de um critério pseudo-aleatório e disparando de maneira compulsória o método de compactação linear da memória.

5 Avaliação Experimental e Resultados

5.1 Configuração do Ambiente de Testes

Os testes empíricos de estresse foram executados sob parâmetros controlados de hardware e software:

- **Processador:** Intel Core i5-10400F (6 núcleos físicos, 12 threads lógicas via *Hyper-Threading*, frequência base de 2.90 GHz).
- **Memória RAM:** 16 GB DDR4 executando a 2666 MHz.
- **Sistema Operacional:** Windows 11 com subsistema Linux (WSL 2 - Ubuntu 24.04.2 LTS).
- **Ambiente de Execução:** OpenJDK 64-Bit Server VM (versão 21.0.11), utilizando parâmetros nativos de gerenciamento de threads POSIX.

O ambiente foi submetido a uma carga padronizada de 100.000 requisições simultâneas com tamanhos de alocação distribuídos de forma uniforme entre 16 e 1024 bytes. O comportamento dinâmico foi testado de forma cruzada em três tamanhos distintos de *heap*: 30 KB, 100 KB e 500 KB, permitindo avaliar o impacto direto da escala do array na eficiência da concorrência.

5.2 Discussão Empírica e Granularidade

A análise dos dados obtidos revela de que forma o conceito de granularidade da tarefa dita as regras de eficiência na programação concorrente. O ganho real de desempenho está associado ao balanço entre o peso aritmético computacional da tarefa e o *overhead* introduzido para controlar e sincronizar a concorrência.

5.2.1 Cenário de Baixa Granularidade (Heap de 30 KB)

No cenário com espaço reduzido de 30 KB, a varredura linear do *Best-Fit* opera sobre uma amostragem de dados reduzida, o que torna a busca sequencial veloz. Como detalhado na Tabela 1, ao injetar paralelismo, o tempo total de execução cai de 502,2 ms para o ponto mínimo de 386,6 ms utilizando 4 threads.

Tabela 1: Métricas de Tempo para o Cenário de Heap de 30 KB.

Configuração de Execução	Tempo Médio (ms)
Sequencial	502,2
Paralelo (2 Threads)	443,2
Paralelo (4 Threads)	386,6
Paralelo (6 Threads)	388,4
Paralelo (8 Threads)	400,2
Paralelo (12 Threads)	380,6
Paralelo (16 Threads)	383,4
Paralelo (24 Threads)	382,8
Paralelo (32 Threads)	386,8

Além da marca de 4 threads, nota-se que o comportamento entra em um patamar plano. O tempo gasto pelas threads disputando o acesso ao monitor exclusivo e as constantes trocas de contexto realizadas pelo escalonador do Sistema Operacional superam e anulam o benefício de segmentar um trabalho computacional leve, gerando um platô ineficiente.

5.2.2 Cenário de Transição (Heap de 100 KB)

A *heap* intermediária de 100 KB funciona como um ambiente de transição comportamental. O tempo de execução sequencial eleva-se para 1.382,2 ms devido ao aumento do espaço amostral de busca linear. Como observado na Tabela 2, o paralelismo gera melhorias iniciais, reduzindo o tempo de processamento para 834,2 ms utilizando 4 threads.

Tabela 2: Métricas de Tempo para o Cenário de Heap de 100 KB.

Configuração de Execução	Tempo Médio (ms)
Sequencial	1.382,2
Paralelo (2 Threads)	970,2
Paralelo (4 Threads)	834,2
Paralelo (6 Threads)	839,0
Paralelo (8 Threads)	995,6
Paralelo (12 Threads)	913,4
Paralelo (16 Threads)	920,6
Paralelo (24 Threads)	906,8
Paralelo (32 Threads)	903,6

No entanto, nota-se uma degradação de performance ao atingir 8 threads (995,6 ms). Esse fenômeno ocorre devido ao aumento na taxa de colisões de gravação. Como o espaço de 100 KB ainda é moderado, múltiplas threads frequentemente localizam o mesmo bloco livre ideal de forma simultânea durante a fase paralela. Ao tentarem gravar na região crítica, causam invalidações sucessivas e disparam laços repetitivos de retentativa, estressando os barramentos de memória cache.

5.2.3 Cenário de Alta Granularidade (Heap de 500 KB)

O ambiente estruturado com 500 KB representa o cenário ideal para a aplicação prática do paralelismo real, caracterizando uma carga predominantemente limitada pela CPU (*CPU-bound workload*). Efetuar buscas com a política *Best-Fit* em um array de tamanho substancial exige esforço matemático e varreduras longas, fazendo com que o algoritmo de thread única sofra com um tempo de execução de 6.480,8 ms.

Tabela 3: Métricas de Tempo para o Cenário de Heap de 500 KB.

Configuração de Execução	Tempo Médio (ms)
Sequencial	6.480,8
Paralelo (2 Threads)	4.023,6
Paralelo (4 Threads)	2.804,8
Paralelo (6 Threads)	2.518,4
Paralelo (8 Threads)	2.687,2
Paralelo (12 Threads)	3.332,0
Paralelo (16 Threads)	3.176,8
Paralelo (24 Threads)	3.116,2
Paralelo (32 Threads)	3.209,8

Como observado na Tabela 3, a divisão do processamento se mostra escalável, reduzindo o tempo de execução para o patamar mínimo de 2.518,4 ms, o que representa um *speedup* de 2,57x. Como as threads passam a maior parte do tempo realizando cálculos aritméticos de busca em seus núcleos isolados, a contenção no momento da gravação torna-se diluída.

O pico de eficiência ocorreu com o uso de 6 threads. Esse teto coincide com o número exato de núcleos de processamento físicos disponíveis no processador i5-10400F utilizado no experimento. A extrapolação dessa marca (de 8 a 32 threads) gera um retrocesso na performance temporal. Isso comprova empiricamente que tecnologias de núcleos lógicos (*Hyper-Threading*) não oferecem vantagens reais para rotinas de varredura linear intensa em arrays primitivos, atuando unicamente como geradoras de sobrecarga em registradores e aumento de latência física.

5.3 Análise Visual Consolidada

Para fins de síntese e mapeamento unificado de escalabilidade, a Figura 1 apresenta a consolidação das curvas e tendências temporais observadas no simulador. O bloco visual expõe a correlação direta entre o número de threads requisitantes e o tempo médio de execução obtido em cada uma das três escalas estruturais configuradas.

A observação das linhas de tendência do gráfico consolida as conclusões empíricas discutidas nas seções anteriores. Nas curvas referentes às *heaps* de 30 KB e 100 KB, fica visualmente claro que o paralelismo atinge um platô prematuro ou até mesmo perda de performance (degradação visível no pico de 8 threads para 100 KB). Isso ilustra o momento exato em que o *overhead* de sincronização, atrelado às colisões de gravação, torna-se mais custoso que o processamento da busca livre.

Em contrapartida, a curva referente ao cenário de 500 KB exhibe o comportamento esperado de escalabilidade real em cargas limitadas por processamento. A linha de tempo de execução desce de forma acentuada até encontrar o seu vale (ponto de maior eficiência) exatamente no eixo de 6 threads. Esse ponto de inflexão espelha perfeitamente a arquitetura do hardware subjacente, comprovando que o ganho máximo converge com os 6 núcleos físicos do processador. A leve subida da curva após essa marca atesta graficamente que o excesso de threads lógicas apenas enfileira requisições no *pipeline* da CPU e aumenta o custo de troca de contexto (*context switch*) pelo Sistema Operacional.

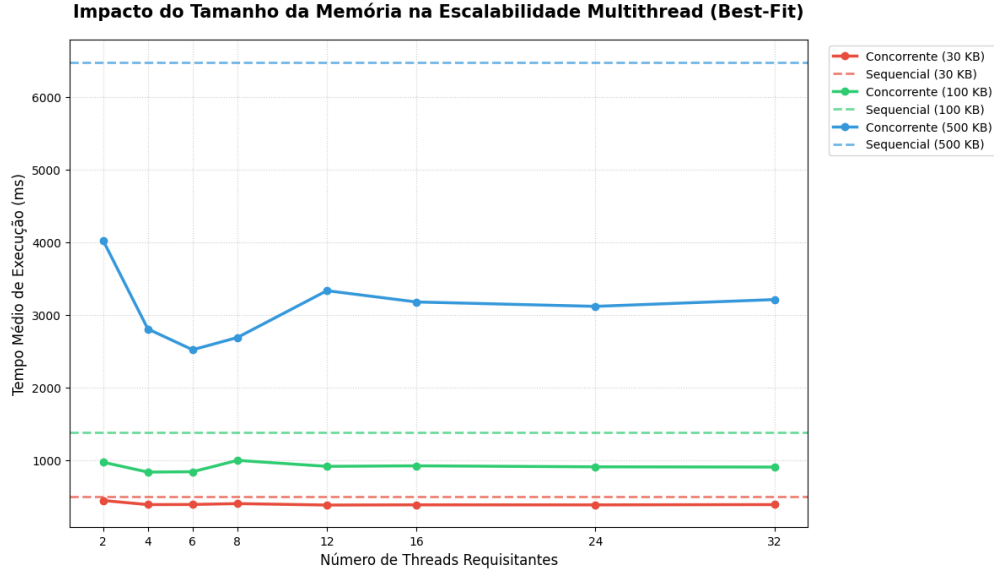


Figura 1: Gráfico consolidado unificado demonstrando o impacto do tamanho da heap e a escalabilidade temporal em função do aumento de threads concorrentes.

6 Conclusões

Este trabalho demonstrou de forma prática o impacto da exclusão mútua e da engenharia paralela aplicada a subsistemas centrais de Sistemas Operacionais. A implementação do simulador dinâmico sob a arquitetura de leitura paralela provou ser uma alternativa arquitetural vantajosa se comparada aos modelos tradicionais de sincronização monolítica, solucionando gargalos de serialização.

Os resultados empíricos consolidam a premissa de que o retorno prático do investimento no desenvolvimento paralelo está atrelado à escala do problema: quanto maior o volume de dados e o esforço de processamento exigido, maior a eficiência da concorrência. A correlação obtida entre o pico de eficiência (6 threads) e as barreiras físicas da CPU reforça os conceitos de afinidade de processador, topologia de cache e limites do hardware.

Como propostas para trabalhos futuros, sugere-se a expansão deste simulador para avaliar estruturas totalmente livres de bloqueios (*Lock-free*) fundamentadas em operações atômicas primitivas de hardware do tipo *Compare-And-Swap* (CAS), ou a investigação do particionamento lógico da *heap* em múltiplas sub-regiões de alocação concorrentes e independentes.

Referências

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- [2] Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
- [3] Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley Professional.
- [4] Berger, E. D., McKinley, K. S., Blumofe, R. D., & Wilson, P. R. (2000). *Hoard: A scalable memory allocator for multithreaded applications*. ACM SIGPLAN Notices, 35(11), 117-128.
- [5] Michael, M. M. (2004). *Scalable lock-free dynamic memory allocation*. ACM SIGPLAN Notices, 39(6), 35-46.
- [6] Leijen, D., Zorn, B., & de Moura, L. (2019). *Mimalloc: Free list sharding in action*. Em Asian Symposium on Programming Languages and Systems (APLAS). Springer.